ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC KINH TẾ

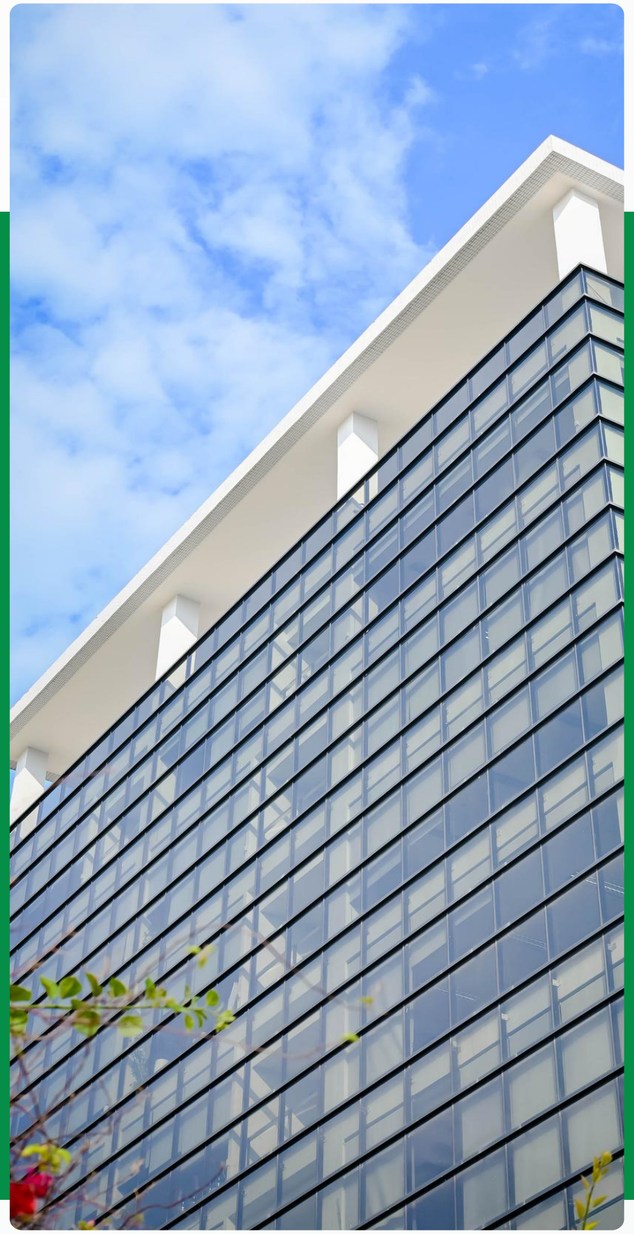
CHƯƠNG 3

QUẢN LÝ DỮ LIỆU VỚI APP ADMIN



LẬP TRÌNH WEB

Khoa Thống kê – Tin học



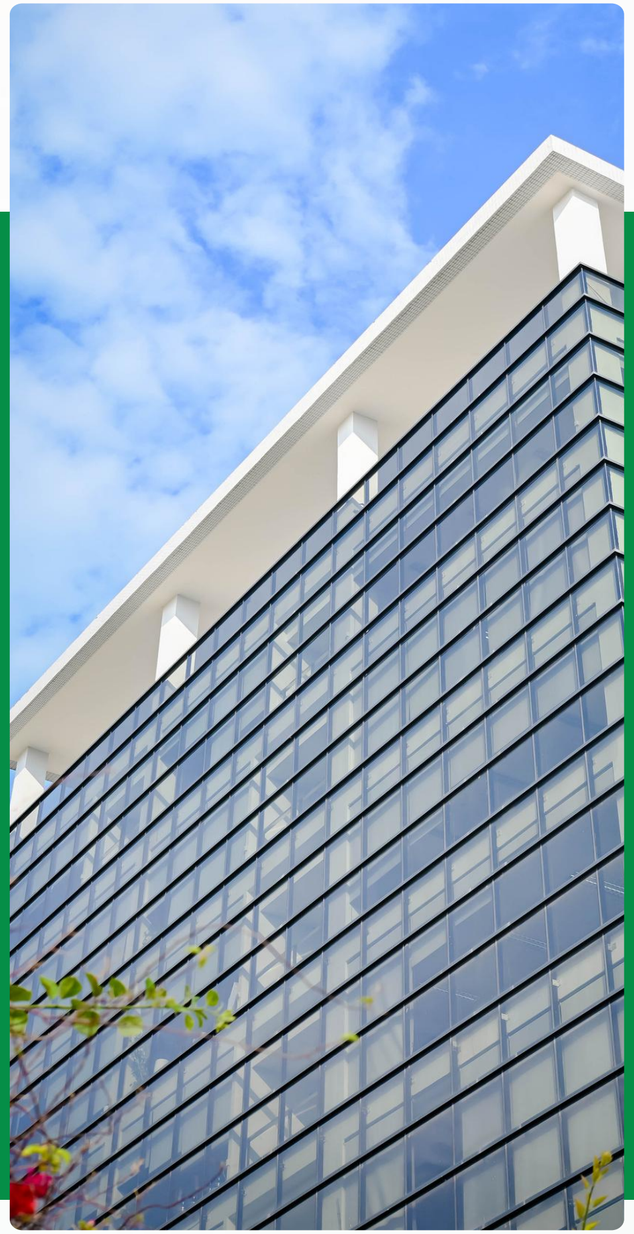
MỤC TIÊU CHƯƠNG

LO1 - Giải thích được chức năng của App Admin trong Django

LO2 - Tạo được tài khoản quản trị và các tài khoản người dùng, nhóm và phân quyền

LO3 - Giải thích được các tùy chỉnh cung cấp bởi lớp AdminSite trong Django

LO4 - Giải thích được các tùy chỉnh có thể thực hiện dùng lớp ModelAdmin của Django



HỌC LIỆU

TL2 - Shaw, B., Badhwar, S., Guest, C., & KS, B. C. (2023), *Web Development with Django: A definitive guide to building modern Python web applications using Django 4*, Packt Publishing Ltd.

3.1 Giới thiệu về Django Admin



Django Admin

- Là một App được xây dựng sẵn của Django
- Cung cấp các giao diện web trực quan để cấp quyền truy cập quản trị vào dữ liệu của Model
- Giao diện quản trị này được thiết kế để quản trị website sử dụng, không phải cho người dùng thông thường
  - Thường có đủ các phần để quản trị được dữ liệu của model thay vì giao diện bắt mắt cho người dùng thông thường
- Có thể được cài đặt sẵn thông qua thiết đặt tại biến INSTALLED_APPS và urlpattens → Truy cập qua URL: **http://Địa_chỉ_website[:cổng]/admin**

```
INSTALLED_APPS = [
'django.contrib.admin',
…
]
```

```
urlpatterns = [
path('admin/', admin.site.urls),
…
]
```

Tạo tài khoản Superuser

- Là tài khoản có quyền cao nhất
  - Sử dụng để quản lý và phân quyền cho các tài khoản và người dùng thông thường
- Tạo bằng lệnh **createsuperuser** của **manage.py**:

```
python manage.py createsuperuser
```

  - Các thông tin cần cung cấp khi tạo tài khoản:
    - Tên tài khoản
    - Địa chỉ email
    - Mật khẩu: phải tuân theo các ràng buộc phổ biến về mật khẩu

Các thao tác quản lý và phân quyền người dùng và nhóm

- Tạo mới người dùng
  - **Lưu ý:** Các tùy chọn về quyền mặc định:
    - **Active**: Được kích hoạt để sử dụng
    - **Staff status**: Được đăng nhập vào trang Admin
    - **Superuser status**: Có toàn quyền quản lý hệ thống
- Truy xuất danh sách người dùng: tìm kiếm và lọc theo loại người dùng
- Cập nhật thông tin người dùng
- Xóa người dùng
- Thêm và phân quyền nhóm
- Gán và bỏ gán nhóm cho người dùng

3.2 Tạo ứng dụng quản lý dữ liệu Model với Admin App



Đăng ký model với App Admin

- Tùy theo loại model, Admin App có thể tạo ra giao diện quản lý dữ liệu Model tương ứng
- Đăng ký (trong tập tin admin.py của mỗi App):

```
from django.contrib import admin
from Tên_app.models import Tên_Model
admin.site.register(Tên_Model)
```

  - URL của trang Admin App quản lý dữ liệu model:

    http://Địa_chỉ_website[:cổng]/admin/Tên_app/Tên_model

Tùy chỉnh giao diện trang Admin

Các tùy chọn thiết đặt của trang Admin

- **site_title**: Điều chỉnh tiêu đề của trang (tag <title>)
- **site_header**: Tiêu đề của trang (hiển thị ở phần đăng nhập)
- **index_title**: Tiêu đề của trang chính (trang liệt kê các Model)
- **index_template**: Đường dẫn đến Template của trang chính, mặc định là admin/index.html
- **app_index_template**: Đường dẫn đến Template của trang chính quản lý từng Model, mặc định là admin/app_index.html
- **login_template**: Đường dẫn đến Template của trang login, mặc định là admin/login.html
- **logout_template**: Đường dẫn đến Template của trang logout, mặc định là registration/logged_out.html
- **password_change_template**: Đường dẫn đến Template của trang đổi mật khẩu, mặc định là registration/password_change_form.html
- **password_change_done_template**: Đường dẫn đến Template của trang hoàn tất đổi mật khẩu, mặc định là registration/password_change_done.html

Tùy chỉnh giao diện trang Admin

Có thể truy xuất và cập nhật các tùy chọn thiết đặt:

- Thông qua đối tượng **site** thuộc module **django.contrib.admin**:

  Ví dụ: *Truy xuất thông tin tùy chỉnh với shell của Django*

```
>>>from django.contrib import admin
>>>admin.site.title_header
```

- Thông qua lớp **AdminSite**: sử dụng tính kế thừa

  Ví dụ: *Thay đổi thông tin tùy chỉnh*

```
from django.contrib.admin import AdminSite
class BookrAdminSite(AdminSite):
  title_header = 'Bookr Admin'
  site_header = 'Bookr administration'
  index_title = 'Bookr site admin'
```

Tùy chỉnh giao diện của từng trang quản lý theo Model

- Sử dụng lớp **ModelAdmin**
- **Tùy chỉnh trang List:**
  - Lựa chọn các cột thông tin: sử dụng thuộc tính **list_display**:

    Ví dụ:

```
class BookAdmin(admin.ModelAdmin):
    list_display = ('title', 'isbn')
```

    Lưu ý: cần điều chỉnh đăng ký lớp admin thành BookAdmin (thay vì admin.ModelAdmin)

```
admin.site.register(Book, BookAdmin)
```

  - Tùy chỉnh hiển thị cho từng cột thông tin: sử dụng phương thức @admin.display()

    Ví dụ:

```
from django.contrib import admin
@admin.display(empty_value="???")
```

Tùy chỉnh giao diện của từng trang quản lý theo Model

Tùy chỉnh trang List (tt):

- Tùy chọn bộ lọc: Sử dụng thuộc tính **list_filter**

  Ví dụ: `list_filter = ('publisher', 'publication_date')`

- Lọc theo từng năm/ tháng/ ngày cụ thể: Sử dụng thuộc tính **date_hierarchy**

  Ví dụ: `date_hierarchy = 'publication_date'`

- Thêm hộp tìm kiếm: Sử dụng thuộc tính search_fields

  Ví dụ: `search_fields = ('title', 'isbn', 'publisher__name')`

Tùy chỉnh giao diện của từng trang quản lý theo Model

Tùy chỉnh trang Add/ Edit:

- Loại bỏ một trường thông tin ra khỏi form nhập: Sử dụng thuộc tính **exclude**

  Ví dụ: `exclude = ('date_edited',)`

- Gộp nhóm các nội dung nhập: Sử dụng thuộc tính **fieldsets**

  Ví dụ:

```
fieldsets = (
("Linkage", {"fields": ("creator", "book")}),
("Review content", {"fields": ("content",
"rating")}),
)
```

ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC KINH TẾ

HẾT CHƯƠNG

KHOA THỐNG KÊ – TIN HỌC

